

06026240 การพัฒนาระบบอัจฉริยะ

LAB 7A

สกัดข้อมูลใบแสดงผลการศึกษา (Transcript)

ด้วย LLM ที่รันบนเครื่องตัวเอง — ออฟไลน์ ฟรี ปลอดภัยตาม PDPA

หัวข้อ	รายละเอียด
กลุ่มเป้าหมาย	นักศึกษาโครงการ "OCR Transcript Pipeline"
ระยะเวลา	120 นาที (ในคาบ) + งานนอกเวลา
เครื่องมือหลัก	Ollama · Typhoon-OCR · Qwen3
ข้อบังคับสำคัญ	ห้ามส่งข้อมูลออกนอกเครื่องโดยเด็ดขาด



อ่านก่อนเริ่มทำแล็บ

Transcript เป็นข้อมูลส่วนบุคคลตาม พ.ร.บ.คุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562
แล็บนี้ทั้งแล็บถูกออกแบบให้รันบนเครื่องของนักศึกษาเอง ไม่มีการเรียก API ภายนอก
ห้ามใช้ Google Colab กับ transcript ของจริง — Colab คือเครื่องของบุคคลที่สาม
ห้ามนำ transcript ของผู้อื่นมาใช้ หากไม่ได้รับความยินยอมเป็นลายลักษณ์อักษร
transcript ตัวอย่างที่ใช้ในแล็บนี้เป็นข้อมูลจำลอง จึงใช้ได้เต็มรูปแบบโดยไม่ต้องปิดทับส่วนใด

1. วัตถุประสงค์การเรียนรู้

เมื่อทำแล็บนี้เสร็จ นักศึกษาจะสามารถ:

- ติดตั้งและเรียกใช้ Vision-Language Model บนเครื่องของตนเองได้ โดยไม่มีค่าใช้จ่ายและไม่ส่งข้อมูลออกนอกเครื่อง
- อธิบายความแตกต่างระหว่าง OCR แบบดั้งเดิม (Tesseract) กับ VLM และเลือกใช้ให้เหมาะกับงาน
- ออกแบบ JSON Schema และใช้ structured output บังคับให้โมเดลตอบตามรูปแบบที่กำหนด
- เขียน prompt ที่ลดการ "เดา" ของโมเดล และให้โมเดลยอมรับได้ว่าอ่านไม่ออก
- ออกแบบกลไกตรวจสอบความถูกต้องภายในเอกสาร (self-verification) โดยไม่ต้องมีเฉลย
- คำนวณและตีความ CER / WER แยกราย attribute

2. ส่วนที่ 1 — ติดตั้งโมเดลบนเครื่องตัวเอง (15 นาที)

i ของที่มีอยู่แล้วจาก Lab ก่อนหน้า ไม่ต้องติดตั้งใหม่

Tesseract พร้อมภาษาไทย · OpenCV · NumPy — ติดตั้งไปแล้วในแล็บก่อนหน้า
virtual environment เดิม — ใช้ตัวเดิมได้เลย ไม่ต้องสร้างใหม่ (เหตุผลอยู่ในหัวข้อ 2.4)
ตรวจว่า Tesseract ยังใช้ได้: `tesseract --list-langs` ต้องเห็นคำว่า tha ในรายการ

2.1 ทำไม่ต้องรันบนเครื่องตัวเอง

1. **ความเป็นส่วนตัว (สำคัญที่สุดสำหรับกลุ่มนี้)** — เมื่อส่งภาพ transcript ไปยัง API ของบริษัทต่างประเทศ ข้อมูลชื่อ-สกุล รหัสนักศึกษา และผลการเรียน จะออกจากประเทศไทยทันที ซึ่งเข้าข่ายการโอนข้อมูลข้ามพรมแดนตาม PDPA มาตรา 28
2. **ค่าใช้จ่าย** — โมเดลที่รันบนเครื่องไม่มีค่าใช้จ่ายต่อครั้ง นักศึกษาทดลองกี่รอบก็ได้ ซึ่งสำคัญมากสำหรับการเรียนรู้
3. **ความสามารถทำซ้ำได้** — ผู้ให้บริการ API เปลี่ยนรุ่นโมเดลเบื้องหลังได้ตลอดเวลาโดยไม่แจ้ง ทำให้ผลการทดลองเมื่อสามเดือนก่อนทำซ้ำไม่ได้ ส่วนโมเดลที่เก็บไว้ในเครื่องจะไม่เปลี่ยน

⚠ ห้ามใช้ Google Colab กับ transcript ของจริง

Colab เป็นเครื่องของ Google ซึ่งเป็นบุคคลที่สาม การอัปโหลด transcript ขึ้นไปคือการส่งข้อมูลส่วนบุคคลออกนอกเครื่อง แม้จะรัน Ollama อยู่บน Colab ก็ตาม

2.2 ติดตั้ง Ollama

Ollama คือโปรแกรมที่ทำให้การรันโมเดลภาษาบนเครื่องง่ายพอ ๆ กับติดตั้งแอปทั่วไป

เมื่อติดตั้งแล้วมันจะเปิดบริการรออยู่ที่ <http://127.0.0.1:11434> โดยเลข 127.0.0.1 คือ loopback address หมายถึง "ตัวเครื่องเอง" ข้อมูลที่ส่งไปที่นี้จะไม่ผ่านการคัดเครือข่ายเลย

ระบบปฏิบัติการ	ขั้นตอน
Windows 10 / 11	เข้า https://ollama.com/download เลือก Windows → ดาวน์โหลด OllamaSetup.exe → ติดตั้ง (ไม่ต้องใช้สิทธิ์ผู้ดูแลระบบ) เมื่อเสร็จจะมีไอคอนที่ system tray มุมขวาล่าง และ service จะเริ่มทำงานอัตโนมัติ
macOS	ดาวน์โหลด Ollama.dmg → ลาก Ollama.app ไป Applications → เปิดครั้งแรก กดยืนยันติดตั้งคำสั่ง command line หรือถ้ามี Homebrew: brew install ollama แล้ว brew services start ollama
Linux / WSL2	curl -fsSL https://ollama.com/install.sh sh บน WSL2 ต้องเปิด service เองในอีก terminal: ollama serve (อย่าปิดหน้าต่างนั้นระหว่างทำแล็บ)

 ตรวจว่า service ทำงานอยู่จริง

เปิดเบราว์เซอร์ไปที่ <http://127.0.0.1:11434> ถ้าเห็นข้อความ "Ollama is running" แสดงว่าพร้อมแล้ว
ถ้าไม่ขึ้น ให้เปิด terminal แล้วสั่ง ollama serve ทิ้งไว้

2.3 ดาวน์โหลดโมเดล 2 ตัว

รวมประมาณ 5.5 GB ควรทำล่วงหน้าเมื่ออยู่บนเครือข่ายที่เร็ว ทั้งสองตัวทำงานคนละหน้าที่และขาดตัวใดตัวหนึ่งไม่ได้

```
# — 1. อ่านภาพเอกสารภาษาไทย → Markdown (~3 GB) —————  
  
# Typhoon-OCR จาก SCB 10X fine-tune จาก Qwen-VL มาสำหรับเอกสารไทย  
  
# ⚠ เป็น task-specific model: รับได้เฉพาะ prompt ของตัวเอง  
  
# สั่งให้พ่น JSON ตาม schema ของเราไม่ได้ จึงต้องมีตัวที่ 2  
ollama pull scb10x/typhoon-ocr1.5-3b  
  
# — 2. Markdown → JSON ตาม schema (~2.5 GB) —————  
  
# เครื่องแรม 8 GB ให้เปลี่ยนเป็น qwen3:1.7b  
# เครื่องแรม 32 GB ขึ้นไป ใช้ qwen3:8b จะได้ผลดีกว่า  
ollama pull qwen3:4b  
  
ollama list      # ตรวจว่าครบทั้งสองตัว
```

 ถ้าชื่อโมเดลเปลี่ยนไป

ชื่อ tag อาจต่างจากที่เขียนไว้ ให้ค้นชื่อล่าสุดที่ <https://ollama.com/search?c=vision>
แล้วตั้งค่าผ่าน environment variable โดยไม่ต้องแก้โค้ด:

export LAB7_MODEL_OCR="scb10x/typhoon-ocr-3b"

2.4 โลยารรี Python — ใช้ environment เดิมของ Lab ก่อนหน้า

✓ ไม่ต้องสร้าง virtual environment ใหม่

Ollama ไม่ใช่แพ็คเกจ Python แต่เป็นโปรแกรมระดับระบบปฏิบัติการที่รันเป็น service แยกต่างหาก

โค้ด Python ของเราเพียงส่งคำขอ HTTP ไปหามันที่ 127.0.0.1:11434 เท่านั้น

โมเดล LLM ทั้งหมดจึงอยู่ "นอก" virtual environment โดยสิ้นเชิง ไม่มีทางชนกับแพ็คเกจใด ๆ ได้

โลยารรีที่ต้องเพิ่มมีเพียง 4 ตัว และไม่มีตัวใดขัดแย้งกับ OpenCV หรือ Tesseract ที่ติดตั้งไปแล้ว

เข้า environment เดิมที่ใช้ใน Lab ก่อนหน้า

source .venv/bin/activate # Windows: .venv\Scripts\activate

pip install requests pymupdf pillow pythainlp

requests → เรียก Ollama ผ่าน HTTP

pymupdf → แปลง PDF เป็นภาพ (import ด้วยชื่อ fitz)

pillow → จัดการภาพ

pythainlp → ตัดคำไทย ใช้ตอนคำนวณ WER

2.5 ตรวจสอบความพร้อมด้วยสคริปต์

```
python3 lab7a_transcript.py --check
```

ผลลัพธ์ที่ควรได้:

✓ พบ Ollama: ollama version 0.x.x

✓ Ollama service ทำงานที่ http://127.0.0.1:11434

✓ [อ่านภาพ] scb10x/typhoon-ocr1.5-3b

✓ [จัด JSON] qwen3:4b

✓ python: fitz / PIL / requests

✓ tesseract (จาก Lab 5-6)

✓ pythainlp

พร้อมใช้งาน ✓

2.6 พิสูจน์ว่ามั่นออฟไลน์จริง

✓ การทดลองที่ต้องทำและบันทึกผล (2 คะแนน)

- รันสคริปต์ให้สำเร็จหนึ่งรอบขณะเชื่อมต่ออินเทอร์เน็ตปกติ เก็บไฟล์ผลลัพธ์ไว้
- ปิด Wi-Fi และถอดสาย LAN ให้เครื่องไม่มีอินเทอร์เน็ตจริง ๆ
- รันสคริปต์ซ้ำด้วยคำสั่งเดิม
- เทียบไฟล์ JSON ทั้งสองรอบ ต้องเหมือนกันทุกตัวอักษร

บันทึกภาพหน้าจอทั้งสองรอบไว้ในรายงาน พร้อมผลของคำสั่ง diff

```
diff output/pred_vlm.json output/pred_vlm_offline.json && echo "เหมือนกันทุกประการ"
```

สาเหตุที่ผลออกมาเหมือนกันเป๊ะ คือเราตั้ง temperature = 0 ทำให้โมเดลเลือก token ที่มีความน่าจะเป็นสูงสุดเสมอ ไม่มีการสุ่ม

2.7 ความต้องการของเครื่อง

สเปกเครื่อง	ชุดโมเดลที่แนะนำ	ความเร็วโดยประมาณ
RAM 8 GB, ไม่มี GPU	typhoon-ocr1.5-2b + qwen3:1.7b	5-10 นาที/หน้า
RAM 16 GB, ไม่มี GPU	ชุดมาตรฐานของแล็บ	1-3 นาที/หน้า
GPU 8 GB VRAM ขึ้นไป	ชุดมาตรฐานของแล็บ	10-30 วินาที/หน้า
Mac (Apple Silicon) 16 GB	ชุดมาตรฐานของแล็บ	20-60 วินาที/หน้า

3. ทำความเข้าใจสามเส้นทางที่จะเปรียบเทียบ

	A — baseline (จาก Lab ก่อนหน้า)	B — vlm (สองชั้น)
วิธีทำงาน	OpenCV → Tesseract → regex	Typhoon-OCR → Markdown → Qwen3 → JSON
จุดแข็ง	เร็วมาก ผลคงที่ ตรวจสอบได้ทุกชั้น	อ่านภาษาไทยแม่นยำกว่ามาก แต่ละโมเดลทำสิ่งที่ตัวเองถนัด
จุดอ่อน	regex พังทันทีเมื่อฟอร์มเปลี่ยน อ่านตารางไม่เก่ง	error สะสมสองชั้น (error propagation) ช้ากว่ามาก
เวลาโดยประมาณ	2-5 วินาที ต่อหน้า	60-180 วินาที ต่อหน้า

3.1 ทำไม Typhoon-OCR ต้องมีชั้นที่สอง



ข้อจำกัดสำคัญที่ต้องเข้าใจ

Typhoon-OCR เป็น task-specific model ผู้พัฒนาระบุชัดว่ามันใช้ได้เฉพาะกับ prompt ของตัวเองเท่านั้น
ถ้าเราสั่งว่า "จงตอบเป็น JSON ตาม schema นี้" มันจะทำได้ไม่ได้ หรือทำได้แย่มาก
มันถูกฝึกมาให้ทำสิ่งเดียว คือ "อ่านหน้าเอกสารออกมาเป็น Markdown ที่รักษาโครงสร้าง"
เราจึงต้องมีโมเดลข้อความอีกตัวมาทำหน้าที่ "จัดรูป" ต่อ

หลักการออกแบบที่ได้จากตรงนี้เรียกว่า **separation of concerns** — ให้แต่ละส่วนของระบบทำสิ่งที่ตัวเองถนัดอย่างเดียว แล้วต่อกันเป็นสายพาน ข้อดีคือเปลี่ยนชิ้นส่วนใดชิ้นหนึ่งได้โดยไม่กระทบส่วนอื่น ข้อเสียคือ error จากชั้นแรกจะไหลลงไปชั้นถัดไปโดยแก้ไม่ได้ (error propagation)

3.2 Structured Output — บังคับรูปร่างของคำตอบ

ปกติเวลาเราขอให้ LLM ตอบเป็น JSON เรามักเขียนในพรอมบ์ว่า "ตอบเป็น JSON เท่านั้น" ซึ่งเป็นเพียง "คำขอ" ไม่ใช่ "การบังคับ" โมเดลอาจใส่คำนำหน้า ใส่ markdown fence ใส่คอมมา ท้ายเกิน หรือถูกตัดกลางคัน

Ollama รองรับการส่ง JSON Schema เข้าไปใน field ชื่อ format ซึ่งจะไปควบคุมขั้นตอนถอดรหัส (constrained decoding) ให้โมเดลเลือกได้เฉพาะ token ที่ยังทำให้ JSON ถูกต้องตาม schema เท่านั้น

```
payload = {  
  "model": "qwen3:4b",  
  "messages": [...],  
  "stream": False,  
  "format": TRANSCRIPT_SCHEMA, # <-- ตรงนี้คือหัวใจ  
  "options": {"temperature": 0},  
}
```

 **สิ่งที่ structured output ทำไม่ได้**

มันรับประกัน "รูปร่าง" ของคำตอบ แต่ไม่รับประกัน "ความจริง" ของเนื้อหา

โมเดลยังคงใส่เกร็ดผิดพลาดในช่องที่ถูกต้องได้อยู่ดี

นี่คือเหตุผลที่เราต้องมีส่วนที่ 4 (ตรวจสอบภายใน) และส่วนที่ 5 (วัดด้วย CER/WER)

3.3 การออกแบบ Prompt เพื่อลดการเดา

พรอมบ์ในสคริปต์มี 6 กติกา แต่ละข้อแก้ปัญหาก็เจอจริง เปิดไฟล์ lab7a_transcript.py ส่วนที่ 4 ประกอบ

กติกา	แก้ปัญหอะไร
ห้ามเดาเกรด ถ้าอ่านไม่ออกให้ตอบ null	LLM ถูกฝึกมาให้ตอบให้ครบเสมอ ถ้าไม่ให้ทางออก มันจะ "ถูกบังคับ" ให้เดา การให้ทางเลือกตอบ null คือการอนุญาตให้มันยอมรับว่าไม่รู้
อ่านทีละแถวจากบนลงล่าง ห้ามข้าม	การตกแถวเป็นความผิดพลาดอันดับหนึ่งของการอ่านตาราง
อธิบายว่า GPA กับ GPS หมายถึงอะไร	ในเอกสารนี้ GPS = เกรดเฉลี่ยประจำภาค ส่วน GPA = เกรดเฉลี่ยสะสม ซึ่งสลับกับความเข้าใจทั่วไป ถ้าไม่บอก โมเดลจะสลับสองค่านี้

กติกา	แก้ปัญหาอะไร
ห้ามคำนวณ GPA/GPS เอง ให้คัดลอกที่พิมพ์ไว้	งานนี้คือการ "ถอดข้อความ" ไม่ใช่การคำนวณ ช่องที่ไม่มีพิมพ์ไว้ให้ใส่ null ห้ามคำนวณขึ้นมาเติม (เหตุผลที่เราไม่เอา GPA มาตรวจย้อนกลับ อยู่ในหัวข้อ 6.1)
ระบุรูปแบบข้อมูลให้ชัด	credit ต้องเป็นจำนวนเต็ม ไม่ใช่ "3(3-0-6)" · วันที่ต้อง YYYY-MM-DD · แปลงเลขไทยเป็นอารบิก
ช่องที่ไม่มีในเอกสาร ให้ใส่ null	ป้องกันการแต่งข้อมูลขึ้นมาเติมช่องว่าง

4. ส่วนที่ 2 — รัน baseline ด้วย Tesseract (15 นาที) (หัวข้อ 4 อันนี้เพื่อใครยังทำแลปก่อนหน้าไม่เสร็จ -ไม่สามารถส่งย้อนหลังได้ แคบทบทวนให้เฉยๆ- ถ้าใครทำเสร็จแล้วก็ข้ามหัวข้อ 4 ไปหัวข้อ 5 ต่อได้เลย)

เราเริ่มจากวิธีดั้งเดิมก่อน ไม่ใช่เพราะมันดี แต่เพราะเราต้องมี "เส้นฐาน" ไว้เปรียบเทียบ หากไม่มีเส้นฐาน เราจะบอกไม่ได้ว่า LLM ดีขึ้นจริงหรือแค่ดูดี

```
cd groupA_transcript
python3 lab7a_transcript.py \
  --input data/transcript_sample.pdf \
  --gt gt/Json_71010001_th.json \
  --pipeline baseline \
  --out output/
```

4.1 อ่านโค้ดในฟังก์ชัน _rule_based_parse

เปิดสคริปต์ที่ส่วนที่ 6 แล้วดู regex ตัวนี้:

```
row_re = re.compile(
    r"(\d{8})\s+({3,80}?)\s+(\d)\s+([ABCD][+]?[WSUIPT])\b",
    re.IGNORECASE,
)
```

5. ส่วนที่ 3 — รัน pipeline VLM (25 นาที)

```
python3 lab7a_transcript.py \  
  
--input data/transcript_sample.pdf \  
  
--gt gt/Json_71010001_th.json \  
  
--pipeline vlm \  
  
--out output/
```

5.1 ตรวจสอบผลลัพธ์กลางทาง — ขั้นตอนที่ห้ามข้าม

สคริปต์จะบันทึกไฟล์ `output/intermediate_vlm.md` ซึ่งคือผลจาก Typhoon-OCR ก่อนถูกส่งเข้าโมเดลข้อความ ให้เปิดดูทุกครั้ง

 **ทำไมต้องดูไฟล์กลางทาง**

เมื่อ JSON ออกมาผิด เราต้องรู้ให้ได้ว่าผิดตั้งแต่ขั้นไหน

ถ้าใน Markdown ก็ผิดอยู่แล้ว → ปัญหาอยู่ที่การอ่านภาพ ให้แก้ที่ DPI หรือคุณภาพภาพต้นฉบับ

ถ้าใน Markdown ถูก แต่ JSON ผิด → ปัญหาอยู่ที่การจัดรูป ให้แก้ที่พารามิเตอร์หรือ schema

การไม่ดูไฟล์กลางทางแล้วไล่แก้พารามิเตอร์ ะ คือการเสียเวลาที่พบบ่อยที่สุด

5.2 การทดลองปรับค่า (บันทึกผลลงตาราง)

ให้ทดลองอย่างน้อย 2 ใน 3 หัวข้อต่อไปนี้ แล้วบันทึกว่าค่า CER เปลี่ยนไปอย่างไร

ตัวแปร	ค่าที่ให้ทดลอง	สมมติฐานที่ต้องตรวจสอบ
ความละเอียดภาพ	LAB7_DPI=100, 150, 300	ยิ่งละเอียดยิ่งแม่นยำหรือไม่ (ลองดูว่าเวลาที่ใช้เพิ่มขึ้นกี่เท่า)
ขนาดโมเดลข้อความ	qwen3:1.7b เทียบ qwen3:4b	โมเดลใหญ่กว่าช่วยที่การอ่านตัวอักษร หรือช่วยที่การจัดโครงสร้าง
ปิด structured output	ลบ fmt= ออกจากการเรียก	JSON เสียบ่อยแค่ไหน เมื่อ parse ได้ เนื้อหาต่างจากเดิมหรือไม่

```
# ตัวอย่างการเปลี่ยนค่าโดยไม่ต้องแก้โค้ด  
  
LAB7_DPI=300 python3 lab7a_transcript.py -i data/x.pdf -g gt/x.json -p vlm -o out300/  
  
LAB7_MODEL_TEXT=qwen3:1.7b python3 lab7a_transcript.py -i data/x.pdf -g gt/x.json -p vlm -o out17/
```

6. ส่วนที่ 4 — ตรวจสอบสอดคล้องภายใน (15 นาที)



แนวคิดหลักของส่วนนี้

ในแล็บเรามีเฉลย จึงตรวจได้ง่าย แต่เมื่อนำระบบไปใช้จริงกับ transcript ใบใหม่ที่ไม่มีใครกรอกเฉลยไว้ เราจะต้องรู้อย่างไรว่าผลถูกหรือผิด

คำตอบคือต้องมี "กฎที่ตรวจกันเองได้ภายในเอกสาร" โดยไม่ต้องพึ่ง ground truth เลย

ส่วนนี้จะสอนทั้งวิธีออกแบบกฎ และที่สำคัญกว่า คือวิธีตัดสินใจว่ากฎแบบไหนควรใช้และแบบไหนไม่ควร

6.1 กฎ 7 ข้อที่สคริปต์ตรวจให้ (def verify_internal(data: dict))

ทุกข้อผ่านเกณฑ์คัดกรองข้างต้น คือถ้าแจ้งเตือนแล้วผิดจริงแน่นอน

กฎ	เนื้อหา	จับความผิดพลาดแบบใด
1	รหัสนักศึกษาเป็นตัวเลข 8 หลัก วันที่อยู่ในรูป YYYY-MM-DD	อ่านเลขเป็นตัวอักษร เช่น 1 เป็น l หรือ 0 เป็น O
2	ปีการศึกษาอยู่ในช่วง 2500–2600 ภาคการศึกษาเป็น 1, 2 หรือ 3	อ่านปี ค.ศ. ปนกับ พ.ศ. หรืออ่านเลขภาคผิด
3	ห้ามมีภาคการศึกษาซ้ำ	โมเดลอ่านหน้าเดิมสองรอบ หรือแบ่งภาคผิด
4	ภาคการศึกษาต้องเรียงจากเก่าไปใหม่	โมเดลสลับลำดับ หรืออ่านข้ามหน้าแล้วย้อนกลับ
5	จำนวนวิชาต่อภาคอยู่ในช่วง 1–12	ภาคที่ไม่มีวิชาเลย = อ่านตกทั้งตาราง ภาคที่มีเกิน 12 วิชา = รวมสองภาคเข้าด้วยกัน
6	รหัสวิชา 8 หลัก · เกรดอยู่ในชุดที่กำหนด หน่วยกิตเป็นจำนวนเต็ม 1–9	อ่านตัวอักษรผิด หรือโมเดลแต่งค่าที่ไม่มีจริงขึ้นมา
7 	ห้ามมีรหัสวิชาซ้ำภายในภาคเดียวกัน	โมเดลอ่านแถวเดิมสองครั้ง — เป็นอาการที่พบบ่อยเมื่อตารางมีเส้นแบ่งไม่ชัด (ลงวิชาซ้ำต่างภาคทำได้ตามระเบียบ จึงไม่ตรวจ)



สังเกตว่าเกรด null ไม่ถือเป็นข้อผิดพลาด

เมื่อโมเดลตอบ null ที่ช่องเกรด แปลว่ามันยอมรับว่าอ่านไม่ออก ซึ่งเป็นพฤติกรรมที่เราต้องการ

สคริปต์จึงนับจำนวนช่องเหล่านี้ไว้รายงาน แต่ไม่นับเป็น issue

ค่านี้เป็นตัวชี้วัดที่มีประโยชน์ในตัวเอง: ถ้าสูงมาก แปลว่าคุณภาพภาพต้นฉบับแย่ ให้ไปแก้ที่ DPI หรือการปรับภาพ

6.2 อ่านผลจากสคริปต์

=====
 ตรวจสอบความสอดคล้องภายใน (กฎเชิงโครงสร้าง — ไม่ใช่ ground truth)
 =====

X vlm: 2 ภาค, 11 วิชา, อ่านเกรดไม่ออก 1 ช่อ่ง

- ลำดับภาคการศึกษาไม่เรียงจากเก่าไปใหม่
 --> โมเดลอาจสลับลำดับ หรืออ่านข้ามหน้า
- [2561/1] รหัสวิชา 13006204 ปรากฏซ้ำในภาคเดียวกัน

--> อาจอ่านแถวเดิมสองครั้ง

ตัวอย่างข้างบนบอกอะไร: โมเดลสลับลำดับภาคการศึกษา และอ่านแถวหนึ่งซ้ำสองครั้ง
 ทั้งสองอย่างเป็นข้อผิดพลาดที่แน่นอน ไม่ใช่การตีความต่างกัน จึงคุ้มค่าที่จะแจ้งเตือน

6.3 รูปแบบที่ควรจดจำและนำไปใช้ในโครงการ

☒ **Generate → Verify → Repair → Escalate**

Generate ให้โมเดลสร้างผลลัพธ์

Verify ตรวจสอบด้วยกฎที่เขียนเอง (7 ข้อข้างต้น)

Repair ถ้าไม่ผ่าน ส่งกลับไปแก้ พร้อมบอกให้ชัดว่าผิดตรงไหน

การบอกแค่ว่า "ผิด ทำใหม่" ไม่ได้ผล เพราะโมเดลไม่รู้ว่ามีอะไร

Escalate ถ้ายังไม่ผ่านสองรอบ ให้ส่งคนตรวจ

ระบบที่ไม่มีทางออกนี้ จะปล่อยข้อมูลผิดออกไปเสมอ

นี่คือรูปแบบที่ใช้จริงในระบบ production ไม่ใช่แค่แบบฝึกหัดในห้องเรียน

7. ส่วนที่ 5 — วัดผลด้วย CER และ WER (25 นาที)

7.1 ทดลองกับโมดูลวัดผลก่อน

```
python3 ../lab7_metrics.py
```

จะเห็นการสาธิตเทียบข้อความ 6 คู่ ให้ดูโดยเฉพาะคู่สุดท้าย ที่ Tesseract อ่านเลข 0 เป็นตัว O ซึ่งเป็นความผิดพลาดคลาสสิกของ OCR

7.2 รันครบทุก pipeline แล้วเปรียบเทียบ

```
python3 lab7a_transcript.py \  
--input data/transcript_sample.pdf \  
--gt gt/Json_71010001_th.json \  
--pipeline all \  
--out output/
```

สคริปต์จะสร้างไฟล์ output/comparison.csv ให้นำไปเปิดใน Excel แล้วสร้างกราฟแท่งเปรียบเทียบ CER แยกตาม attribute และ pipeline

Attribute	N	CER	Exact	Ins	Miss	Hall

ประเภทวิชา	12	n/a	91.7%	6	0	1
สรุปผลการศึกษา	5	0.8333	80.0%	5	0	1

11. แหล่งอ้างอิงและเอกสารเพิ่มเติม

หัวข้อ	แหล่งข้อมูล
Ollama	https://ollama.com/download https://docs.ollama.com/capabilities/structured-outputs
Typhoon OCR	https://ollama.com/scb10x https://opentyphoon.ai/blog/en/typhoon-ocr-release
Qwen3-VL	https://ollama.com/library/qwen3-vl
Tesseract	https://tesseract-ocr.github.io/tessdoc/
PyThaiNLP	https://pythainlp.org/
PDPA	พ.ร.บ.คุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562 มาตรา 4, 24, 28